

Scrum:

- Features are written from the perspective of a user (user story)
 - Collection of these is a product backlog (wishlist)
- The features chosen for a release are called the release backlog
 - Estimates amount of work involved for each user story
- Complete features for release using sprints
 - Short duration milestones where they should be ship ready
- Burndown chart
 - Day by day measure of the amount of work remaining in a given sprint/release
- Daily scrum
 - Daily stand up meetings listing work completed since last meeting
 - Sprint retrospective: what went well, what didn't
- Product owner
 - Ensures right features are in the product backlog
- Scrum Master
 - Ensures project is running smoothly, project manager
- Developer
 - Builds product
- Tester
 - Tests product

Sdlc:

1. Planning and analysis (requirement analysis)
 - a. Business and user requirements about final product
 - b. Cost, time estimations and scoping
2. Designing
 - a. Create high level diagrams
3. Development
 - a. Write the code
4. Testing
 - a. Review product for bugs
5. Deployment
 - a. Product release
6. Maintenance
 - a. Continuous updates and support

Waterfall

- Each stage starts only after the previous one is finished
- Not flexible

Agile

- Lots of on-going sprints/cycles
- Produce only essential docs (just good enough)

- Lean methodology
 - 3 steps, build product, measure feedback and success, learn how to improve
- Extreme programming
 - Test-driven development
 - Code refactoring
 - Pair programming

DevOps

- Different teams work together
- Ensures existing processes are optimized and streamlined
- Continuous integration/continuous delivery
 - Teams are ready for deployment at any moment, uses centralized repository
 - Product can be updated continuously, many times a day if needed
- Introduces automations

Code review

- Have an open mind
- Read the description
- Skim through the changes

Pull request

1. Define a clear goal and stick to it
2. Write a clear, concise, descriptive title
3. Add a tl;dr
4. Give context (why you added it, what it does)
5. instructions on how to test
6. Link to existing issues if possible
7. Use graphics, recordings and github styles if helpful
8. Write tests if possible
9. Review PR yourself first

Design patterns

Extreme programming

Unit testing

- Small test for one feature
- Can be automated so that code gets pushed only when all tests passed
- Write a test case for each input (code coverage)

TDD

1. Write tests
2. Run the tests (must fail)
3. Write the actual code (simple as you can)

4. Make all the tests pass
5. Refactor and improve

S - single responsibility

O - open for extension, closed for modification

L - subclasses must be swappable for base

I - interface segregation (many small interfaces are better than one chunky one)

D - dependency inversion (high level depends on abstractions)